

Processes, Threads, and Goroutines

1. Processes (The Heavyweight)

A **Process** is a running program. If you open Google Chrome, you have launched the Chrome Process.

- Every process gets its own massive, isolated chunk of the computer resources (called "**Context.**")
- It gets its own **Virtual Address Space** (a private sandbox of memory), its own **Stack** and **Heap**, and its own **Program Counter** (tracking which instruction is executing).
- **The Problem:** Because a Process is so heavy, switching between them (a **Context Switch**) takes a long time. The OS has to save the state, pause the program, load the state of the next process, and start it.

2. Threads (The Lightweight Process)

To solve the slow context switching of Processes, operating systems introduced **Threads**.

- A Thread lives inside a Process. A single Process can have many threads.
- **Shared Context:** Threads share the heavy stuff (like the Virtual Memory sandbox and the Heap) with their parent Process.
- **Unique Context:** Threads only keep their own lightweight stuff (their own Stack and their own Program Counter).

Because they share memory, switching from **Thread A** to **Thread B** is faster than switching between two full Processes. Modern **OS** handle concurrency by scheduling Threads to run on the CPU cores, not Processes.

3. Goroutines (The Featherweight)

Threads are still considered "heavy" in modern development. A standard **Java** or **C++** Thread takes about **1-2 MB** of RAM. If a web server tries to launch **10,000** threads to handle **10,000** user requests, the computer will run out of memory and crash. Go solves this brilliantly with **Goroutines**.

- **What is a Goroutine?** A Goroutine is a function executing concurrently with other Goroutines. It is often described as a "**Green Thread**" or an "**Application-Level Thread.**"
- **The Magic of the Go Scheduler:** Instead of relying on the Operating System to manage thousands of heavy threads, Go launches just one OS Thread per CPU Core.
- The **Go Runtime Scheduler** sits on top of these OS threads. It creates thousands of microscopic Goroutines (only take about 2 KB of RAM) and seamlessly juggles them back and forth across the few available OS Threads.

Expert Takeaway

In **Node.js** (single thread) or **Java/Python** (use OS threads), handling **100,000** simultaneous WebSocket connections requires complex architectural gymnastics and massive server clusters. In Go, you can literally just launch **100,000** Goroutines on a single basic server, and the Go Runtime Scheduler will flawlessly manage all of them in the background with barely any memory overhead!